SSO Account Recovery Circuits Audit

OpenZeppelin Security - May 15, 2025

Summary

Summary

Type: Account Abstraction
Timeline: March 24, 2025 → April 11, 2025
Languages: Circom

Findings
Total issues: 9 (9 resolved)
Critical: 0 (0 resolved) · High: 0 (0 resolved) · Medium: 2 (2 resolved) · Low: 3 (3 resolved)

Notes & Additional Information
4 notes raised (4 resolved)

Scope

OpenZeppelin audited the Moonsong-Labs/zksync-social-login-circuit repository at commit 27cda6e. In scope were the following files:

```
 ├── jwt-tx-validation.circom
└── utils
    ├── array.circom
    ├── base64url-to-base64.circom
    ├── bytes-to-field.circom
    ├── bytes.circom
    ├── constants.circom
    ├── fields.circom
    ├── jwt-data.circom
    ├── jwt-verify.circom
    ├── replace-all.circom
    ├── verify-nonce.circom
    └── verify-oidc-digest.circom
```

System Overview

Account recovery is widely used in web2 and is arguably an essential feature to ensure a seamless experience for the end user. Bringing this feature to blockchain accounts using zero-knowledge proofs aligns with Matter Labs' goal of creating a unified web3 experience.

The audited codebase, developed by Moonsong Labs in collaboration with Matter Labs, consists of Circom circuits used to validate the ownership of an account associated with a Single Sign-On (SSO) provider. The codebase will initially provide support for Google accounts. This validation of account ownership can notably be used for the recovery of smart wallet accounts, allowing a wallet owner to register a Google account as a recovery method, and later use it to recover access to their wallet. This kind of account recovery method is already used extensively in web2, and could, for example, offer a secure and user-friendly solution in case of loss of private keys. Using zero-knowledge proofs provides the additional benefit of confidentiality for use on public blockchains, as the link between the Google account and the address can be kept private.

The protocol for account recovery described above is made up of two parts:

1. On-chain smart contracts (e.g. `OidcValidator`) that are in charge of keeping a registry of Google's RSA public key, managing the logic to register an account and later recover it. This recovery is done by calling a Groth16 verifier, which validates a zero-knowledge argument for the satisfiability of an arithmetic circuit.

2. This arithmetic circuit is implemented and compiled using Circom, a Domain-Specific Language (DSL). More details about the circuit logic are given in the section below.

The `.circom` files represent the circuit described in 2. above and make up the scope of the current audit. The smart contracts mentioned in 1. are considered out of scope for this report and will be audited as part of a separate engagement.

SSO, OIDC, and JWTs

The codebase under review uses a number of standards and protocols (SSO, OIDC, and JWTs) which are briefly described below.

SSO

At a high level, SSO serves as an architectural pattern where users can authenticate once to access multiple services. In the present system, this architecture relies on two key technical standards: OpenID Connect (OIDC), which standardizes the protocol framework, and JSON Web Tokens (JWTs), which provide the cryptographic payload structure.

OIDC

OIDC extends OAuth 2.0 by standardizing how information about the identity of users flows between systems. When a user authenticates with an identity provider, OIDC generates a JWT containing cryptographically signed claims about the user's identity. This JWT becomes the portable proof of authentication that third-party services (in this case, the account recovery protocol) can verify independently.

JWT

JWTs are compact, self-contained tokens that are used to securely transmit claims between parties. Their structure consists of three base64url-encoded parts separated by dots: a header, a payload, and a signature (`header.payload.signature`). The header is a JSON containing information about the algorithm used to generate the signature, while the payload is a JSON containing "claims" (statements about the issuer of the JWT, the user, and metadata like expiration time). Finally, the signature provides authentication and data integrity protection to the JWT.

High-Level Description of the Recovery Process

The recovery process takes place in two main steps:

1. **Account Linking**: Users register their Google account with their smart wallet through the `OidcValidator` smart contract. To ensure that no information about which Google account was used is leaked, this is done by submitting a `digest`. This `digest` is the Poseidon hash of `iss || aud || sub || salt`, where `iss`, `aud`, and `sub` are JWT claims respectively identifying the issuer (Google), the audience (who the JWT is for), and the subject (the user).

   The `salt` is a secret field element that is generated by a *salt service*. The default salt service runs on a server and generates the salt pseudorandomly but deterministically based on a JWT

and a secret static seed. The user will need to be able to reproduce this same salt during the account recovery phase, which is why determinism is important. In theory, Google could figure out the issuer, audience, and subject: adding a secret salt to the Poseidon hash prevents Google from identifying the account from this digest.

2. **Recovery Proof**: When recovering their account, a user interacts with the `OidcValidator` contract through an auxiliary account. To avoid potential issues with account recovery replay and validate the link between the Google account and the auxiliary address, the user generates a `nonce` in the following manner:

    1. Calculate the *nonce content*, `content = Keccak256(auxiliaryAddress || autoIncrementalValue || timeLimit)`, where

        - `auxiliaryAddress` is the address of the new auxiliary account, from which the user is trying to recover their old account.
        - `autoIncrementalValue` is a per-account nonce that increments each time the Verifier smart contract receives a valid proof.
        - `timeLimit` is the expiry date of the recovery process.

    2. Split the 256-bit nonce content into two field elements: `content[0]` and `content[1]`, representing strings of 31 bytes and 1 byte, respectively.
    3. Generate a secret field element `blindingFactor` off-chain. This value, which is used once, prevents Google from linking the user's smart wallet to their Google account.
    4. Calculate `nonce = Poseidon(content[0], content[1], blindingFactor)`.

After generation, this `nonce` is sent as an additional user-controlled field of the JWT and signed by the identity provider (Google). The `OidcValidator` contract then calls the zk verifier, providing Google's RSA public key, the digest, and the nonce content as public inputs.

Circuit Logic

The circuit has multiple responsibilities:

1. Validating that the signature sent in the JWT is associated with the public key and is correct. This validates that the JWT was signed using RSA-SHA256 by the identity provider, Google.
2. Extracting the `iss`, `aud`, and `sub` fields from the JWT to reconstruct the digest and validate it against the `digest` given as public input. This validates that the wallet owner did link the Google account for whom the JWT was signed to its wallet.
3. Recomputing the `nonce` from the public input and the `blindingFactor` given as private input, and validating that it is equal to the `nonce` extracted from the JWT. This ensures that the Google account validated the `nonce`, and, thus, notably the auxiliary address.

Security Model and Trust Assumptions

During the audit, the following trust assumptions were made:

- The RSA public keys sent as public input are valid and provide enough bits of security.
- The only identity provider supported is Google. As part of the audit, the parameters were validated and the codebase was tested against JWTs generated following Google's standards. If more providers were to be supported in the future, some security considerations have been provided for such a scenario in the recommendations section.

Medium Severity

Non-Determinism of Some Inputs

Some input signals were found to be under-constrained:

- The bit sizes of the `nonceLength`, `issLength`, `audLength`, and `subLength` input signals are not constrained. While `LessThan` is applied [1] [2] to these values, it is possible for them to overflow the number of bits set as the parameter. This could lead to `LessThan` returning wrong values. For example, `nonceLength` could be replaced by p−kp-kp − k for a small kkk, which could make the nonce returned by `ExtractNonce` have leading and/or trailing 0s. The same is true for the other lengths and their associated fields as well. While an immediately exploitable security concern was not observed here, it is nonetheless a dangerous and avoidable pattern. Consider range-checking these lengths with a `Num2Bits` to avoid potential issues.
- The `nonce` extracted from the JWT payload is assumed to be encoded with base64url. The nonce is thus decoded, replacing the ASCII characters `45` ("-") with `43` ("+"), `95` ("_") with `47` ("/"), and `0` ("NULL") with `61` ("="). However, as the nonce is a user-controlled input, it is possible for some characters in the nonce to already be base64-encoded. For example, the nonces `abc-` and `abc+` would lead to the same base64url-decoded nonce, introducing non-determinism. Note that concrete security concerns were not observed to arise from this, and validating the strict base64url encoding of `nonce` would likely be too expensive. Thus, simply consider documenting this behavior.
- The `nonce` is a 32-byte value that is base64url-decoded from the JWT payload. These 32 bytes are packed in the `packedNonce` signal, equal to `nonce[31] + 2 ** (8) * nonce[30] + ... + 2 ** (8 * (31)) * nonce[0]`. However, it is possible for this value to exceed `p` and overflow, for example, if `nonce[0] = 255`. While a concrete scenario exploiting this non-determinism was not observed, consider nonetheless range-checking the nonce to ensure that it is a proper field element. Alternatively, consider documenting this potential overflow.

Consider addressing the identified instances with additional range checks or documentation to mitigate the risk of issues arising from them. While no concrete attacks were identified, non-determinism should be avoided when possible and documented otherwise.

Update: *Resolved in pull request #40 at commit 5d02de9. All the issues identified were resolved. A new* `AssertFitsBinary` *function was added to range-check the lengths and address the first point. The* `nonce` *is now validated not to contain "+" or "/" characters, which are not valid base64url characters, resolving the second point. New functions were added to detect overflows when computing the nonce, addressing the concern from the third point.*

Mismatched Base64url Decoding May Break Completeness

RFC-7519 standardizes the concept of a JWT, describing it as "a sequence of URL-safe parts separated by period ('.') characters. Each part contains a **base64url**-encoded value". However, the circuit decodes the payload as if it were base64-encoded, not base64url-encoded. If the payload were to contain "-" or "_", the circuit would reject such inputs as they do not correspond to valid base64 characters. However, these are valid characters in a base64url encoding. This makes it theoretically possible to have valid Google OIDC JWTs which would be unverifiable by the circuit, breaking completeness.

Note that during testing, the audit team was unable to produce such JWTs under normal constraints (e.g., having only ASCII characters in email addresses, `nonce` *correctly set as the base64url encoding of a 32-byte hash and other Google-compatible fields). However, the tests were non-exhaustive, and the payload should be base64url-decoded to reduce the risk of issues.*

Consider calling `Base64UrlToBase64` before decoding the payload, similarly to how it is done for the nonces, to avoid potential completeness issues.

Update: *Resolved in pull request #40 at commit 0548942. Payloads are now correctly base64url-decoded.*

Low Severity

Length Calculation Unintentionally Iterated

In `verify-oidc-digest.circom`, the length of the intermediate signal array `packedIss` is given as `computeIntChunkLength(issFieldLength)`. However, `issFieldLength` is the length of `iss` in field elements, not bytes, and is previously calculated as `var issFieldLength = computeIntChunkLength(maxIssLength)`. The final result is that the length of the array is computed as `computeIntChunkLength(computeIntChunkLength(maxIssLength))` instead of `computeIntChunkLength(maxIssLength)`. In this case, they happen to be equal since `computeIntChunkLength(computeIntChunkLength(maxIssLength)) = computeIntChunkLength(1) = 1`.

In fact, there is an `assert` statement in line 33 constraining `issFieldLength == 1`. Therefore, this error cannot represent a vulnerability to the circuit in its current form. However, this calculation is not done as intended, so it still merits correction.

Consider replacing the in-line calculation of the length of the array with the pre-calculated `issFieldLength` (i.e., `signal packedIss[issFieldLength] <== PackBytes(maxIssLength)(iss)`).

Update: *Resolved in pull request #40 at commit 0548942.*

Naming Conventions

Many of the circuits in the zksync-social-login-circuits repository do conversions between different data types represented as native field elements. As such, a value inside the circuit may represent either

- a bit
- a byte
- a base64 character
- a base64url character
- an ASCII character
- a native field element
- a chunk of a big integer, for RSA

Since, ultimately, the circuit checks against an expected hash, input validation for each of these data types, in the form of range checks within the circuit, is omitted. It is common for sub-circuits in circom to be written this way (i.e., as promise algorithms) to reduce constraints when composed with other circuits in a larger circuit.

The larger circuit ensures security since it will fail to produce a correct proof if any of the inputs to a sub-circuit is not of the promised form (say, for example, if a field element that is expected to be a byte is given a value which is outside the range of 0-255). However, the absence of explicit range checks makes it even more important to establish a naming convention to designate the expected "type" of data being held by a given native field element in order to avoid confusion and make the code self-documenting.

To address this, consider implementing the following recommendations.

Add Suffixes to Each Circuit Value or Array of Values Indicating the Expected Data Type

For example,

- `sha[256]` could be `shaBits[256]`
- `FindRealMessageLength` could be `FindRealMessageLengthAscii` or something similar.
- `blindingFactor` could be `blindingFactorField`.
- `pubkey[k]` could be `pubkeyChunks[k]`.

Add a Suffix to Each Length Indicating the Unit of Measurement

For example, in `jwt-verify.circom`,

- `messageLength` could be `messageLengthAscii`.

- `rsaMessageSize` could be `rsaMessageSizeChunks`.

Replace "decode" With "(convert) to ASCII"

Since ASCII is a type of encoding as well, adopting this phrasing makes the output more explicit.

Indicate Big-endian Interpretation

In `BytesToField`, big-endian interpretation is used but not explicitly stated. The name could be changed to `BytesToFieldBigEndian` or something similar, and the documentation could include the interpretation used.

Use Descriptive Names

Most names throughout the codebase are descriptive, but there are a few cases where more descriptive names would be beneficial. For example,

- `n` can be `numBitsPerChunk` or something similar.
- `k` can be `numChunks`.
- `gts` can be `comparisonResults`, `greaterThans`, or something similar.

Establish a Naming Convention for Constants

`MAX_NONCE_BASE64_LENGTH` follows the pattern `[BOUND]_[OBJECT]_[UNIT]_LENGTH` whereas `MAX_LENGTH_DECODED_NONCE` follows the pattern `[BOUND]_LENGTH_[UNIT]_[OBJECT]`. Consider establishing a naming convention with a consistent order that specifies the unit of measurement of different array lengths throughout the code. In this example, the two lengths could be renamed to `MAX_LENGTH_NONCE_BASE64` and `MAX_LENGTH_NONCE_ASCII`, respectively.

Update: *Resolved in pull request #40 at commit 0548942. The Matter Labs team renamed most constants, variable names, and signals according to our suggestions. Where there were exceptions, the purpose was to maintain consistency with the notation of the zkemail dependency, and/or the meaning of the name is in the documentation.*

Dangerous SHA-256 Padding Length

The circuit constraints the character `46` ("." in ASCII) to be unique in `message`. Besides, `message` is padded according to the SHA-256 specifications, meaning that it is padded with a single bit `1`, followed by enough 0s to be congruent to 448 modulo 512, followed by its length as a 64-bit number.

However, it is theoretically possible for this length to include a "46" character, which would break completeness as the "46" character would no longer be unique. The tests conducted by the audit team indicate that given the constraints on the lengths of the input, this is not currently possible.

However, if the `maxMessageLength` parameter were to be increased in the future, this issue would need to be re-investigated to avoid potential completeness issues.

Consider documenting this constraint. Alternatively and to be more future-proof, consider ensuring that a `46` byte appearing in the length does not break completeness.

Update: *Resolved in pull request #40 at commit 0548942. The period character is now validated to be unique using the new* `CountCharOccurrencesUpTo` *function, which ignores the SHA-256 padding.*

Notes & Additional Information

Redundant Computation

In `verify-oidc-digest.circom`, three parameters are computed using the `computeIntChunkLength` function of `@zkemail/circuits/utils/bytes.circom`:

```
  var issFieldLength = computeIntChunkLength(maxIssLength);
 var audFieldLength = computeIntChunkLength(maxAudLength);
 var subFieldLength = computeIntChunkLength(maxSubLength);
```

However, in lines 37-39, they are computed once again in-line to give the length of three corresponding arrays:

```
  signal packedIss[computeIntChunkLength(issFieldLength)] <==
PackBytes(maxIssLength)(iss);
 signal packedAud[computeIntChunkLength(maxAudLength)] <==
PackBytes(maxAudLength)(aud);
 signal packedSub[computeIntChunkLength(maxSubLength)] <==
PackBytes(maxSubLength)(sub);
```

Consider simplifying the code by using the pre-computed values directly inside the brackets in lines 37-39.

Update: *Resolved in pull request #40 at commit 0548942.*

Missing or Misleading Documentation

Throughout the codebase, multiple instances of missing or misleading documentation were identified:

- In lines 22-23 of `jwt-tx-validation.circom`, `maxMessageLength`, and `maxB64PayloadLength` could be documented as being lengths that are denominated in base64url and ASCII characters, respectively.
- In line 4 of `bytes-to-field.circom`, consider documenting the `BytesToField` template. This NatSpec should also document the assumption that the `inputs` array contains valid bytes (i.e., field elements in [0, 255]), and that the output can overflow.
- In line 13 of `constants.circom`, the KID is documented as being "20 bytes long", while the `JWT_KID_LENGTH` function returns 40. Consider clarifying the comment to indicate the reason for this behavior.
- In line 49 of `jwt-verify.circom`, consider documenting the fact that while the `AssertZeroPadding` template makes the assumption that `startIndex - 1` fits in `ceil(log2(maxArrayLen))` bits which is not respected if `messageLength` is 0, the `LessThan` would still work as expected.
- Similarly, in the fields.circom file, the `ExtractNonce`, `ExtractIssuer`, `ExtractAud`, and `ExtractSub` templates use the `RevealSubstring` as a sub-template with the same argument for `maxSubstringLength` and `substringLength` (e.g. for nonce: `nonceKeyLength`). This contradicts the assumption in `SelectSubArray` that `length` is assumed to fit in `ceil(log2(maxArrayLen))` bits, as, for example, the nonce technically `substringLength = 8` does not fit in `ceil(log2(substringLength)) = 3` bits. Consider documenting that this is a known behavior, and that the `GreaterThan` still works as expected.
- In the `fields.circom` file, consider documenting in the `ExtractNonce`, `ExtractIssuer`, `ExtractAud`, and `ExtractSub` templates that `nonceKeyStartIndex` is implicitly validated to be in the correct range (i.e., [0, 1023]) by the `RevealSubstring` and `VarShiftLeft` templates.
- In line 18 of `verify-oidc-digest.circom`, the documentation states that the `expectedDigest` input is provided by the user. However, it is provided by the `OidcValidator` smart contract.
- In line 32 of `verify-nonce.circom`, there is an input array documented as `txHash[2]` which appears as `content[2]` in the code.
- In line 32 of `verify-nonce.circom`, it is stated that `txHash` is provided as an array of length 2 since it represents a Keccak-256 hash and the field modulus is 254-bit long. However, it does not indicate how the bits are split between the two values. For example, two plausible splits are 31 bytes and 1 byte, or 16 bytes and 16 bytes. This changes the semantic meaning of the nonce content, as the nonce is calculated as `Poseidon(3)(content[0], content[1], blindingFactor)`. As Poseidon is a field hashing algorithm and does not begin by concatenating values, this introduces some ambiguity about how the reader should expect the nonce to be calculated.
- The provided protocol documentation does not indicate how the time limit associated with the recovery data is used to calculate the nonce content hash.

Consider adding missing comments and revising the aforementioned instances to improve consistency and more accurately reflect the implemented logic, making it easier for auditors and other parties examining the code to understand what each section of code is designed to do.

Update: *Resolved in pull request #40 at commit 0548942. The Matter Labs team implemented our suggestions, except in a few cases where they perceived that documenting how a particular edge case is covered could be confusing. After discussing with them, we agree with their assessment.*

Code Quality Improvements

Throughout the codebase, multiple opportunities for code improvement were identified:

- The `n` and `k` parameters must be chosen so that n*k > 2048 for RSA-2048 and n < 127. In addition, `maxMessageLength` and `maxB64PayloadLength` must be multiples of 64 and 4, respectively. Consider adding asserts to check these assumptions during code generation to avoid misconfigurations.
- The `JWT_TYP_LENGTH` function returns 11 but should return 12 to be consistent with its documentation (or its documentation is incorrect). Moreover, a lot of the functions in `constants.circom` are unused and could be removed.
- There is an inconsistent number of spaces used for tabs between the files. For example, `jwt-data.circom` uses two spaces, whereas, `fields.circom` uses four.
- At the beginning of `verify-nonce.circom`, the `MAX_LENGTH_DECODED_NONCE()` and `MAX_BYTES_FIELD()` constant functions are defined, returning 33 and 32, respectively. Since their lengths are off by one, a constraint is added in line 48 which ensures that `nonce[32] === 0`. However, the 32 index represents a one-element subarray proceeding from the index `MAX_BYTES_FIELD()` and ending at index `MAX_LENGTH_DECODED_NONCE() - 1`, inclusive. Consider changing this to a `for` loop constraining each entry of this subarray to be 0, and changing the hardcoded `32` to `MAX_BYTES_FIELD()`. This will make the code consistent with the rest of the code in the file, and allow the protocol to be easily extensible in the future.
- The circuit takes `expectedIss` and `expectedAud` as private inputs. However, these are only used for validation as private inputs do not contribute to the security of the system. Consider removing them to simplify the code. Alternatively, if such validation is desired, consider adding an `expectedSub` to also validate the `sub` field of the JWT.

Consider addressing the identified instances to improve the quality of the code.

Update: *Resolved in pull request #40 at commit 0548942. All the identified points were addressed.*

Typography Errors

Throughout the codebase, multiple instances of typographical errors were identified:

- In line 27 of `base64url-to-base64.circom`, "from a base64" should be "from a base64url".

Additionally, the JWT is the original base64URL-encoded string. Consider rephrasing the sentence for clarity.

- In line 13 of `fields.circom`, the "maxNonceLength" parameter in the NatSpec should be "maxPayloadLength".
- In line 16 of `jwt-data.circom`, "index o '"nonce":'" should be " index of '"nonce":'".
- In lines 18-22 of `jwt-data.circom`, ".." should be "." at the end of the lines.
- In line 24 of `verify-nonce.circom`, "Our nonce are" should be either "Our nonces are" or "Our nonce is".
- In line 27 of `verify-nonce.circom`, "Decide" should be "Decode".
- In line 31 of `verify-nonce.circom`, "to prevent google to identify" should be "to prevent Google from identifying".
- In line 7 of `verify-oidc-digest.circom`, "Recalculate oidc_digest and checks matches with provided one." should be "Recalculates oidc_digest and checks that it matches the one provided."
- In lines 11-13 of `verify-oidc-digest.circom`, "Max length if characters" should be "Max length in characters".
- In lines 14-17 of `verify-oidc-digest.circom`, the first letter after each input name should be capitalized.
- In line 17 of `bytes.circom`, the `maxLength` parameter is defined as an "@input" in the NatSpec, when it should be a "@param".
- In line 33 of `jwt-tx-validation.circom`, "Index for "nonce":" substring" should be "Index for '"nonce":' substring".
- In line 45 of `jwt-verify.circom`, the word "bits" should be added to the end of the sentence.

To improve the readability of the codebase, consider addressing the above instances.

Update: *Resolved in pull request #40 at commit 1e0e35d.*

Recommendations

Security Considerations in Case of Protocol Changes

The following recommendations are made to help in case changes are made to the recovery protocol:

- The audit considered potential completeness issues arising from insufficient lengths (e.g. `maxIssLength` being too short), but did not find any such issue. However, as stated in the trust assumptions section, it was assumed that the only supported provider is Google. Notably, the `sub` parameter could, in theory, have a length of up to 255 ASCII characters, far above the current circuit limit of 31. If more providers are planned to be supported in the future, consider validating those length parameters to ensure that completeness is maintained.
- A second recommendation regarding the length of the SHA-256 padded message was changed to a low-severity issue following discussions with the client.

Conclusion

The codebase under audit introduces circuits for proving the ownership of Google accounts through zero-knowledge proofs for the purpose of smart account recovery. The circuits validate the signature associated with the JWT and parse its payload to extract the information required to validate the digest and the nonce. This design notably provides confidentiality, hiding the link between a smart account and its associated Google accounts.

The audit did not reveal any major issues. Some completeness and non-determinism concerns were identified and recommendations were provided to further improve the quality of the code. Overall, the implementation was found to be sound and well-written. The Matter Labs and Moonsong Labs teams are appreciated for being responsive and providing the audit team with extensive project documentation.

Zama

ZKsync

Services

Smart Contract Security Audit

Blockchain Infrastructure Audit

ZKP Audit

Security Services

Solutions

Ecosystem Stack

Financial Institutions Stack

Resources

Research

Documentation ↗

Stats

Forum ↗

Ethernaut CTF ↗

© 2026 Zeppelin Group Ltd